ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO



BÁO CÁO ĐỒ ÁN HỌC PHẦN

PBL3: MẠNG MÁY TÍNH – NHÓM 7

ĐỀ TÀI:

PGP

NGUYÊN LÝ RSA VÀ MÔ HÌNH MÃ HÓA LAI RSA – AES

Giảng viên hướng dẫn: TS. Trần Thị Minh Hạnh

Sinh viên thực hiện (Nhóm 7):

STT	Họ và tên	MSSV
1	Vũ Duy Dương	106230007
2	Huỳnh Thiện Lộc	106230052
3	Phan Vinh Sơn	106230059

Đà Nẵng, năm 2026

MỤC LỤC

1. Danh mục thuật ngữ và viết tắt 3

2. Giới thiệu đề tài 5
2.1. Bối cảnh kỹ thuật . . . 5
2.2. Mục tiêu . . . 6
2.3. Phạm vi . . . 6

3. Vì sao PGP sử dụng mã hóa lai? 8
3.1. Nguyên lý kết hợp . . . 8

4. Quy trình Hybrid Encryption 9
4.1. Sơ đồ tuần tự tương tác . . . 9

5. Cơ sở toán học của thuật toán RSA 10
5.1. Mô hình toán học tổng quát (Lý thuyết) . . . 10
5.2. Chứng minh tính đúng đắn qua Định lý Euler . . . 10

6. Cài đặt RSA bằng Python 12
6.1. Bảng đặc tả các hàm chính . . . 12
6.2. Mã nguồn giải thuật Euclid mở rộng (`algorithms/euclid.py`) . . . 12
6.3. Mã nguồn thuật toán RSA (`algorithms/rsa.py`) . . . 13
6.4. Kết quả thực thi (`algorithms/rsa.py`) . . . 15

7. Ví dụ số học RSA – Tạo khóa, mã hóa và giải mã 17
7.1. Khởi tạo hệ thống và tính toán khóa . . . 17
7.2. Thực nghiệm mã hóa và giải mã . . . 17
7.3. Chuỗi chuyển đổi dữ liệu và khóa . . . 18

8. Tích hợp RSA và AES trong Hybrid Encryption 19
8.1. Cấu trúc khung gói tin (Packet Framing & Offset) . . . 19
8.2. Cơ sở toán học của AES-GCM (AEAD) . . . 19
8.3. Phân công nhiệm vụ hai phía . . . 20
8.4. Luồng điều khiển của mô-đun tích hợp . . . 20
8.5. Mã nguồn tích hợp Hybrid PGP (`algorithms/hybrid_pgp.py`) . . . 21

9. Demo chương trình và kết quả kiểm thử 23
9.1. Kết quả thực thi trên terminal . . . 23

10. Mô phỏng truyền thông PGP Hybrid qua Socket TCP 25
10.1. Kiến trúc chương trình . . . 25
10.2. Đóng gói và truyền nhận gói tin . . . 25
10.3. Bắt tay và xác thực Public Key . . . 26
10.4. Quy trình hoạt động . . . 26
10.5. Kịch bản kiểm thử trên mạng . . . 27
10.6. Hướng dẫn chạy . . . 27
10.7. Kết quả thực nghiệm . . . 27

11. Kết quả đạt được và hướng phát triển 29
11.1. Tổng kết khối lượng công việc . . . 29
11.2. Kế hoạch triển khai giai đoạn tới . . . 29

TÀI LIỆU THAM KHẢO **31**

1. Danh mục thuật ngữ và viết tắt

Để văn phong báo cáo cô đọng, trong toàn bộ tài liệu các thuật ngữ mật mã học và mạng máy tính được phân loại và chuẩn hóa theo danh mục dưới đây:

Ký hiệu / Thuật ngữ	Phân loại	Tên tiếng Anh	Ý nghĩa chuyên môn
PGP	Tiêu chuẩn	Pretty Good Privacy	Giao thức mã hóa bảo vệ quyền riêng tư qua kênh mạng mở.
RSA	Bất đối xứng	Rivest – Shamir – Adleman	Hệ mật mã khóa công khai dựa trên độ khó bài toán phân tích số nguyên lớn.
AES	Đối xứng	Advanced Encryption Standard	Chuẩn mã hóa đối xứng khối tốc độ cao, dùng ma trận thay thế hoán vị SPN.
Hybrid Encryption	Kiến trúc	Mã hóa lai	Mô hình kết hợp AES (mã hóa dữ liệu) và RSA (bảo vệ khóa phiên).
Session Key	Khóa	Khóa phiên	Khóa đối xứng ngẫu nhiên chỉ có giá trị cho một phiên truyền thông duy nhất.
Public / Private Key	Khóa	Khóa công khai / Bí mật	Cặp khóa bất đối xứng: chia sẻ công khai để mã hóa / giữ bí mật để giải mã.
PKESK	Gói tin PGP	Public-Key Encrypted Session Key	Gói tin chứa Session Key đã được mã hóa bằng khóa công khai người nhận.
SEIPD	Gói tin PGP	Symmetrically Encrypted Integrity Protected Data	Gói tin chứa dữ liệu đã được mã hóa đối xứng kèm cơ chế kiểm tra toàn vẹn.
AES-GCM	Chế độ AES	Galois/Counter Mode	Chế độ mã hóa xác thực (AEAD) cung cấp đồng thời tính bí mật và toàn vẹn.
IV / Nonce	Tham số AES	Initialization Vector / Number Used Once	Vector khởi tạo ngẫu nhiên (12 bytes) giúp triệt tiêu hiện tượng trùng lặp bản mã khi mã hóa cùng dữ liệu.

Authentication Tag	Toàn vẹn	Message Authentication Code (MAC)	Mã kiểm thực (16 bytes) giúp phía nhận phát hiện mọi hành vi can thiệp hay sửa đổi bản mã trên mạng.
OAEP	Cơ chế đệm	Optimal Asymmetric Encryption Padding	Cơ chế đệm ngẫu nhiên an toàn chống tấn công chọn bản mã cho RSA.
Miller–Rabin	Số học	Miller–Rabin Primality Test	Thuật toán kiểm tra số nguyên tố xác suất, được dùng để sinh p, q động trong quá trình tạo khóa RSA.
TCP	Giao thức mạng	Transmission Control Protocol	Giao thức truyền luồng tin cậy, có thứ tự, bảo đảm dữ liệu đến đủ và đúng thứ tự.
Socket	Lập trình mạng	Berkeley Sockets API	Điểm cuối giao tiếp giữa hai tiến trình qua mạng, định danh bởi cặp địa chỉ IP và cổng.
Length-prefixed Framing	Kỹ thuật	Đóng khung tiền tố độ dài	Kỹ thuật ghi 4 byte độ dài gói trước dữ liệu để bộ nhận xác định đúng ranh giới thông điệp trên luồng TCP.
Fingerprint	Xác thực khóa	SHA-256 Fingerprint	Mã băm SHA-256 của Public Key, dùng đối chiếu ngoài kênh nhằm phát hiện khóa bị thay thế.

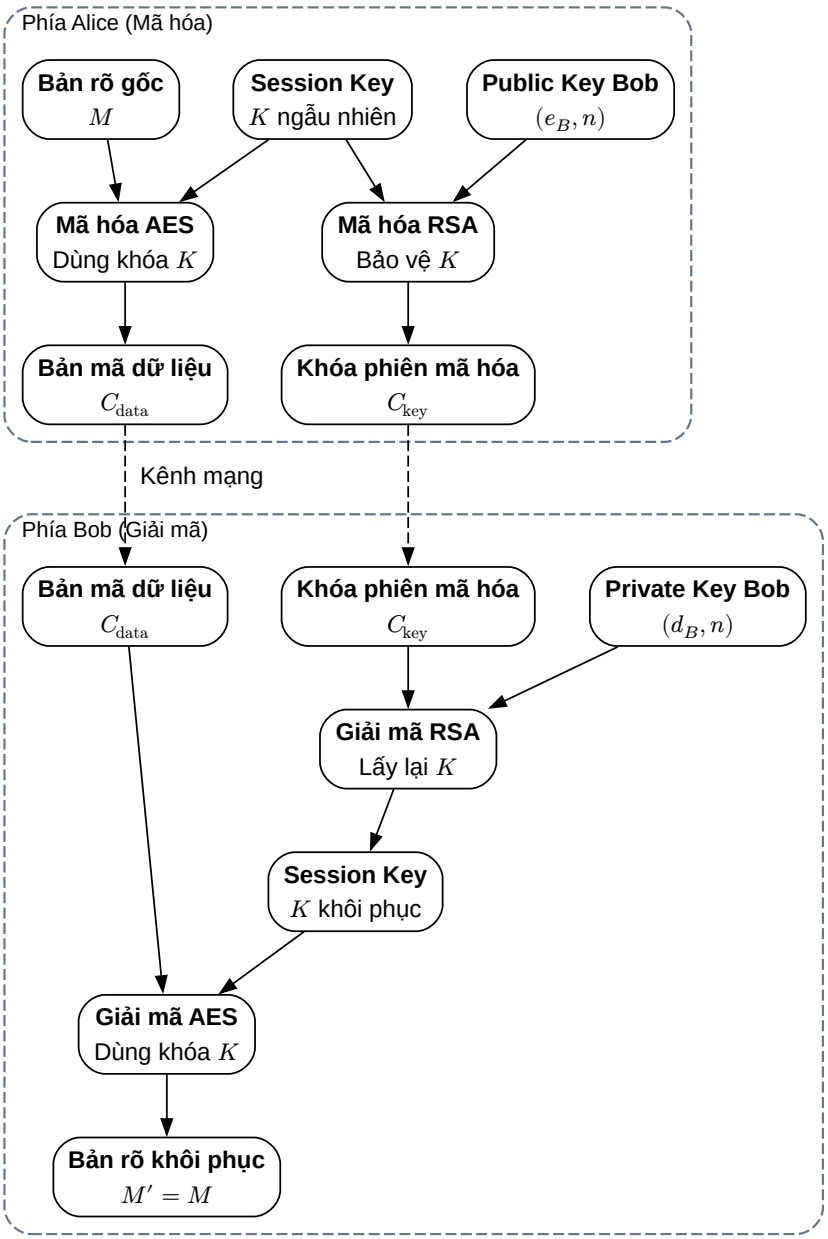
2. Giới thiệu đề tài

2.1. Bối cảnh kỹ thuật

Truyền thông an toàn trên mạng đòi hỏi đảm bảo đồng thời ba thuộc tính: tính bí mật, toàn vẹn và xác thực [1]:

- **Mã hóa đối xứng (AES):** Xử lý nhanh, hiệu quả với dữ liệu lớn nhưng phụ thuộc vào bài toán thỏa thuận khóa bí mật qua kênh truyền [2].
- **Mã hóa bất đối xứng (RSA):** Giải quyết việc trao đổi khóa bằng cặp Public/Private Key, nhưng tốc độ xử lý chậm do chi phí tính lũy thừa mô-đun lớn [3].

Chuẩn OpenPGP [4], [5] giải quyết sự đánh đổi này bằng kiến trúc **Hybrid Encryption**: dùng AES mã hóa dữ liệu và dùng RSA bảo vệ khóa phiên. Trong mô hình truyền thông mật mã học chuẩn mực, hai thực thể trao đổi tin được quy ước định danh là **Alice (phía gửi tin)** và **Bob (phía nhận tin)**. Bob công khai Public Key (e_B, n) để mọi người gửi thông điệp mã hóa cho mình, trong khi giữ bí mật Private Key (d_B, n) để giải mã.



Hình 1: Sơ đồ nguyên lý mã hóa và giải mã lai RSA – AES khép kín trong mô hình PGP

2.2. Mục tiêu

Chuẩn OpenPGP hoàn chỉnh chứa nhiều cơ chế phức hợp (hệ thống gói tin đa tầng, chứng thực Web of Trust, nén dữ liệu). Trong phạm vi đề tài môn học, nhóm tập trung thực nghiệm mô hình cốt lõi:

1. Nắm vững cơ sở số học của RSA (thuật toán Euclid mở rộng, định lý Euler).
2. Cài đặt thuật toán RSA bằng Python thuần để quan sát quá trình tính toán.
3. Tích hợp module RSA với AES-GCM tạo thành luồng mã hóa lai.
4. Thiết kế các kịch bản kiểm thử nhằm đánh giá tính đúng đắn và khả năng bắt lỗi toàn vẹn.

2.3. Phạm vi

- **Thực hiện:** Cài đặt Textbook RSA dạng số học, kết hợp thư viện chuẩn AES-GCM để gửi và nhận thông điệp có kiểm thực tag.

- **Chưa thực hiện:** Chưa áp dụng cơ chế đệm an toàn RSA-OAEP [3], chưa xây dựng mạng lưới tín nhiệm Web of Trust và cấu trúc nhị phân RFC 4880.

3. Vì sao PGP sử dụng mã hóa lai?

Mã hóa lai giải quyết bài toán dung hòa giữa tốc độ xử lý và khả năng phân phối khóa an toàn:

AES – Mã hóa đối xứng	RSA – Mã hóa bất đối xứng
Dùng chung một Session Key bí mật.	Dùng cặp Public Key và Private Key.
Tốc độ phần cứng cực nhanh, tối ưu cho dữ liệu lớn.	Tốc độ chậm do tính lũy thừa mô-đun trên số nguyên lớn.
Cần kênh an toàn để trao đổi khóa phiên.	Cho phép gửi khóa an toàn qua kênh công khai.
Không mã hóa được nếu chưa có khóa chung.	Không tối ưu để mã hóa trực tiếp khối dữ liệu dài.

3.1. Nguyên lý kết hợp

Trong kịch bản truyền thông giữa Alice (người gửi) và Bob (người nhận):

- AES chịu trách nhiệm mã hóa toàn bộ dữ liệu người dùng bằng khóa ngẫu nhiên K do Alice sinh ra.
- RSA chịu trách nhiệm mã hóa duy nhất khóa K bằng Public Key của Bob (e_B). Chỉ có Bob sở hữu Private Key (d_B) tương ứng mới có thể giải mã lấy lại K.

Công thức đóng gói dữ liệu của PGP:

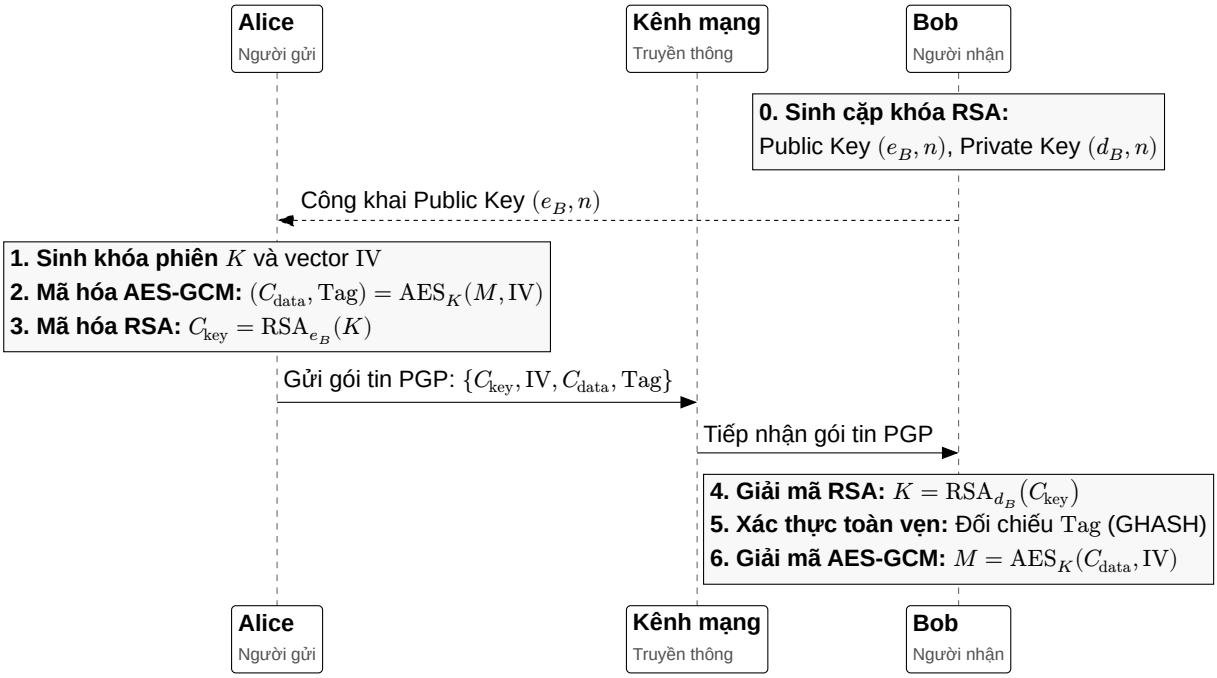
$$\text{PGP Packet} = \left\{ \text{RSA}_{e_B}(K), \quad \text{AES}_K(M) \right\}$$

Kết luận: AES bảo vệ dữ liệu, RSA bảo vệ khóa phiên.

4. Quy trình Hybrid Encryption

Phía Alice (Mã hóa)	Phía Bob (Giải mã)
1. Nhận Public Key (e_B, n) của Bob từ kênh công khai. 2. Sinh Session Key (K) và vector khởi tạo (IV). 3. Dùng AES-GCM mã hóa: $(C_{\text{data}}, \text{Tag}) = \text{AES}_K(M, \text{IV})$. 4. Dùng Public Key Bob: $C_{\text{key}} = \text{RSA}_{e_B}(K)$. 5. Ghép gói tin $\{C_{\text{key}}, \text{IV}, C_{\text{data}}, \text{Tag}\}$ và gửi qua mạng.	1. **Khởi tạo:** Bob sinh cặp khóa RSA (e_B, n) & (d_B, n), công khai (e_B, n). 2. Nhận gói tin PGP từ Alice. 3. Dùng Private Key: $K = \text{RSA}_{d_B}(C_{\text{key}})$. 4. Đối chiếu Tag để xác thực toàn vẹn bản mã. 5. Dùng K và IV giải mã: $M = \text{AES}_K(C_{\text{data}}, \text{IV})$.

4.1. Sơ đồ tuần tự tương tác



Hình 2: Sơ đồ tuần tự quy trình truyền thông mã hóa lai PGP giữa Alice và Bob

5. Cơ sở toán học của thuật toán RSA

Thuật toán RSA [6] dựa trên tính bất đối xứng tính toán của bài toán phân tích thừa số nguyên tố: việc nhân hai số nguyên tố lớn rất dễ dàng về mặt tính toán, nhưng việc tìm lại hai thừa số đó từ tích của chúng là bài toán khó giải quyết trong thời gian đa thức.

5.1. Mô hình toán học tổng quát (Lý thuyết)

Mô hình hệ mật mã RSA bao gồm 3 giải thuật hình thức:

1. **Sinh cặp khóa (Key Generation):**
   - Chọn ngẫu nhiên hai số nguyên tố lớn phân biệt p và q.
   - Tính modulus $n = p \times q$. Không gian khóa và bản mã được xác định trên vành số nguyên mô-đun $\mathbb{Z}_n$.
   - Tính hàm phi Euler: $\varphi(n) = (p-1)(q-1)$.
   - Chọn số mũ công khai e thỏa mãn điều kiện:

$$1 < e < \varphi(n) \quad \text{và} \quad \gcd(e, \varphi(n)) = 1$$

   - Tính số mũ bí mật d là phần tử nghịch đảo nhân của e theo mô-đun $\varphi(n)$:

$$e \times d \equiv 1 \bmod \varphi(n)$$

   - Khóa công khai gồm cặp (e, n), khóa bí mật tương ứng gồm cặp (d, n).

2. **Mã hóa (Encryption):**
   - Bản rõ biểu diễn dưới dạng số nguyên $M \in \mathbb{Z}_n$ ($0 \leq M < n$).
   - Bản mã C được tính theo phép lũy thừa mô-đun:

$$C = M^e \bmod n$$

3. **Giải mã (Decryption):**
   - Từ bản mã C, khôi phục lại bản rõ M' bằng khóa bí mật:

$$M' = C^d \bmod n$$

5.2. Chứng minh tính đúng đắn qua Định lý Euler

Thuật toán giải mã RSA đảm bảo khôi phục chính xác thông điệp gốc dựa trên cơ sở lý thuyết số học sau:

1. **Điều kiện biên không gian bản rõ:** Giá trị số học của thông điệp thỏa mãn $M \in \mathbb{Z}_n$ ($0 \leq M < n$).
2. **Quan hệ giữa cặp số mũ:** Do $ed \equiv 1 \bmod \varphi(n)$, tồn tại số nguyên $k \geq 1$ sao cho:

$$ed = 1 + k\varphi(n) = 1 + k(p-1)(q-1)$$

3. **Áp dụng Định lý Euler:** Với mọi M nguyên tố cùng nhau với n ($\gcd(M, n) = 1$):

$$M^{\varphi(n)} \equiv 1 \bmod n$$

4. **Biến đổi giải mã:**

$$M' = C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n = M^{1+k\varphi(n)} \bmod n$$

$$M' = M \times \left(M^{\varphi(n)}\right)^k \bmod n \equiv M \times 1^k \bmod n \equiv M \bmod n$$

Do $0 \leq M < n$, giá trị thu được sau giải mã M' đồng nhất hoàn toàn với bản rõ ban đầu M.

> **Khuyến nghị tiêu chuẩn:**
> Mô hình trên là Textbook RSA. Tiêu chuẩn RFC 8017 [3] yêu cầu kích thước khóa tối thiểu 2048-bit và bắt buộc kết hợp cơ chế đệm an toàn (RSA-OAEP) để chống các kỹ thuật tấn công chọn bản mã.

6. Cài đặt RSA bằng Python

Thuật toán RSA được tổ chức thành các mô-đun độc lập trong thư mục `algorithms/` nhằm phục vụ việc kiểm thử từng thành phần:

6.1. Bảng đặc tả các hàm chính

Tên hàm	Mô tả chức năng & Giải thuật
`extended_gcd(a, b)`	Cài đặt giải thuật Euclid mở rộng: đệ quy tìm ước chung lớn nhất và các hệ số Bézout x, y sao cho $ax + by = \gcd(a, b)$. Dùng để tính số mũ bí mật d.
`is_prime(n, k)`	Kiểm tra số nguyên tố xác suất bằng thuật toán Miller–Rabin với k vòng thử (mặc định $k = 20$), dùng nguồn ngẫu nhiên an toàn `random.SystemRandom`. Là cơ sở để sinh p, q động.
`generate_prime(bits)`	Sinh ngẫu nhiên một số nguyên tố có độ dài `bits` bit: bật bit cao nhất và bit thấp nhất để bảo đảm độ dài, sau đó lặp tới khi vượt qua `is_prime`.
`generate_rsa_keys(p=None, q=None, e=65537, bits=256)`	Giai đoạn sinh khóa: nếu truyền `p, q` thì dùng trực tiếp (ví dụ minh họa $p = 61, q = 53, e = 17$); nếu không, tự sinh hai số nguyên tố p, q động (mặc định 256-bit) thỏa $\gcd(e, \varphi(n)) = 1$, rồi tính $n = p \times q$, $\varphi = (p-1)(q-1)$, $d =$ `mod_inverse(e, phi)`, trả về cặp `((e, n), (d, n))`.
`rsa_encrypt(pub_key, M)`	Giai đoạn mã hóa: Áp dụng công thức $C = M^e \bmod n$ sử dụng hàm lũy thừa modulo tối ưu `pow(M, e, n)`.
`rsa_decrypt(priv_key, C)`	Giai đoạn giải mã: Áp dụng công thức $M = C^d \bmod n$ thông qua `pow(C, d, n)`.

6.2. Mã nguồn giải thuật Euclid mở rộng (`algorithms/euclid.py`)

Python

```python
"""
Mô-đun giải thuật Euclid mở rộng và tính nghịch đảo modulo.
Cung cấp nền tảng toán học để tìm khóa bí mật d trong RSA: e * d ≡ 1 (mod phi).
"""

def extended_gcd(a, b):
    """
    Giải thuật Euclid mở rộng tìm ước chung lớn nhất gcd(a, b)
    và các hệ số Bézout x, y sao cho: a*x + b*y = gcd(a, b).
    """
    if a == 0:
        return b, 0, 1
```

```
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def mod_inverse(e, phi):
    """
    Tìm phần tử nghịch đảo modulo d = e^(-1) mod phi.
    Ném lỗi ValueError nếu e và phi không nguyên tố cùng nhau.
    """
    gcd, x, _ = extended_gcd(e, phi)
    if gcd != 1:
        raise ValueError(f"Không tồn tại nghịch đảo modulo: gcd({e}, {phi}) = {gcd} != 1")
    return (x % phi + phi) % phi

if __name__ == "__main__":
    e = 17
    phi = 3120
    d = mod_inverse(e, phi)
    print(f"Kiểm thử Euclid mở rộng:")
    print(f"e = {e}, phi(n) = {phi}")
    print(f"Khóa bí mật d = {d}")
    print(f"Kiểm tra: ({e} * {d}) mod {phi} = {(e * d) % phi}")
```

6.3. Mã nguồn thuật toán RSA (`algorithms/rsa.py`)

Python

```
"""
Mô-đun thuật toán RSA số học.
Cài đặt quá trình kiểm tra nguyên tố Miller-Rabin, sinh khóa, mã hóa và giải mã số học.
"""

import random
from .euclid import extended_gcd, mod_inverse


def is_prime(n, k=20):
    """Kiểm tra số nguyên tố xác suất bằng thuật toán Miller-Rabin."""
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False

    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
```

```
        d //= 2

    rng = random.SystemRandom()
    for _ in range(k):
        a = rng.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True


def generate_prime(bits=128):
    """Sinh ngẫu nhiên một số nguyên tố có độ dài bit chỉ định."""
    rng = random.SystemRandom()
    while True:
        candidate = rng.getrandbits(bits) | (1 << (bits - 1)) | 1
        if is_prime(candidate):
            return candidate


def generate_rsa_keys(p=None, q=None, e=65537, bits=256):
    """
    Sinh cặp khóa Public Key (e, n) và Private Key (d, n).
    - Nếu truyền p, q cụ thể: dùng trực tiếp p, q đó (ví dụ minh họa p=61, q=53, e=17).
    - Nếu không truyền p, q: tự động sinh 2 số nguyên tố lớn để modulus n đủ chứa khóa AES-128.
    """
    if p is None or q is None:
        prime_bits = max(bits // 2, 72)
        while True:
            p = generate_prime(prime_bits)
            q = generate_prime(prime_bits)
            if p != q and extended_gcd(e, (p - 1) * (q - 1))[0] == 1:
                break

    n = p * q
    phi = (p - 1) * (q - 1)
    d = mod_inverse(e, phi)
    return (e, n), (d, n)


```

```python
def rsa_encrypt(public_key, plaintext_num):
    """
    Mã hóa số học RSA: C = M^e mod n.
    """
    e, n = public_key
    if plaintext_num >= n:
        raise ValueError(f"Dữ liệu ({plaintext_num}) phải nhỏ hơn modulus n ({n})")
    return pow(plaintext_num, e, n)


def rsa_decrypt(private_key, ciphertext_num):
    """
    Giải mã số học RSA: M = C^d mod n.
    """
    d, n = private_key
    return pow(ciphertext_num, d, n)


if __name__ == "__main__":
    pub_key, priv_key = generate_rsa_keys(p=61, q=53, e=17)
    M = 125
    C = rsa_encrypt(pub_key, M)
    M_prime = rsa_decrypt(priv_key, C)
    print("Kiểm thử module RSA:")
    print(f"Public Key (e, n): {pub_key}")
    print(f"Private Key (d, n): {priv_key}")
    print(f"Bản rõ: {M} -> Bản mã: {C} -> Khôi phục: {M_prime}")
    assert M == M_prime, "Lỗi giải mã!"
    print("Khôi phục thành công.")
```

6.4. Kết quả thực thi (`algorithms/rsa.py`)

Kết quả xuất ra khi chạy `python3 -m algorithms.rsa` (chế độ minh họa truyền thống với $p = 61, q = 53, e = 17$):

```
Public Key (e, n): (17, 3233)
Private Key (d, n): (2753, 3233)
Bản rõ: 125 -> Bản mã: 1516 -> Khôi phục: 125
Khôi phục thành công.
```

Điểm cần nhớ:

- Cả hai hàm `rsa_encrypt` và `rsa_decrypt` đều thực thi thuật toán tính lũy thừa theo mô-đun (`pow(base, exp, mod)`), chỉ khác nhau ở số mũ: quá trình mã hóa sử dụng số mũ công khai e, trong khi quá trình giải mã sử dụng số mũ bí mật d.
- Khi **không** truyền `p, q` , hàm `generate_rsa_keys` mặc định sinh khóa động 256-bit (`bits=256` , `e=65537`). Độ dài này bảo đảm modulus $n > 2^{128}$, đủ lớn để mã hóa trọn vẹn một Session Key AES-128 (16 bytes) mà không bị mất bit.

7. Ví dụ số học RSA – Tạo khóa, mã hóa và giải mã

Để minh họa nguyên lý toán học của hệ mật mã RSA, phần này thực nghiệm tính toán trên không gian số nhỏ có thể kiểm tra từng bước bằng tay.

7.1. Khởi tạo hệ thống và tính toán khóa

1. **Chọn hai số nguyên tố nhỏ:**

$$p = 61, \quad q = 53$$

2. **Tính Modulus hệ thống n:**

$$n = p \times q = 61 \times 53 = 3233$$

3. **Tính giá trị hàm Euler $\varphi(n)$:**

$$\varphi(n) = (p-1)(q-1) = 60 \times 52 = 3120$$

4. **Chọn số mũ công khai e:** Chọn $e = 17$ thỏa mãn $1 < e < 3120$ và $\gcd(17, 3120) = 1$.
5. **Tính số mũ bí mật d bằng thuật toán Euclid mở rộng:** Tìm d sao cho $17 \times d \equiv 1 \bmod 3120$:

$$17 \times 2753 = 46801 = 15 \times 3120 + 1 \Rightarrow d = 2753$$

Loại khóa	Thành phần	Giá trị số học
Public Key	(e, n)	$(17, 3233)$
Private Key	(d, n)	$(2753, 3233)$

7.2. Thực nghiệm mã hóa và giải mã

Giả sử Alice muốn truyền thông điệp số $M = 125$ ($M < n$) cho Bob:

- **Mã hóa (Alice thực hiện):**

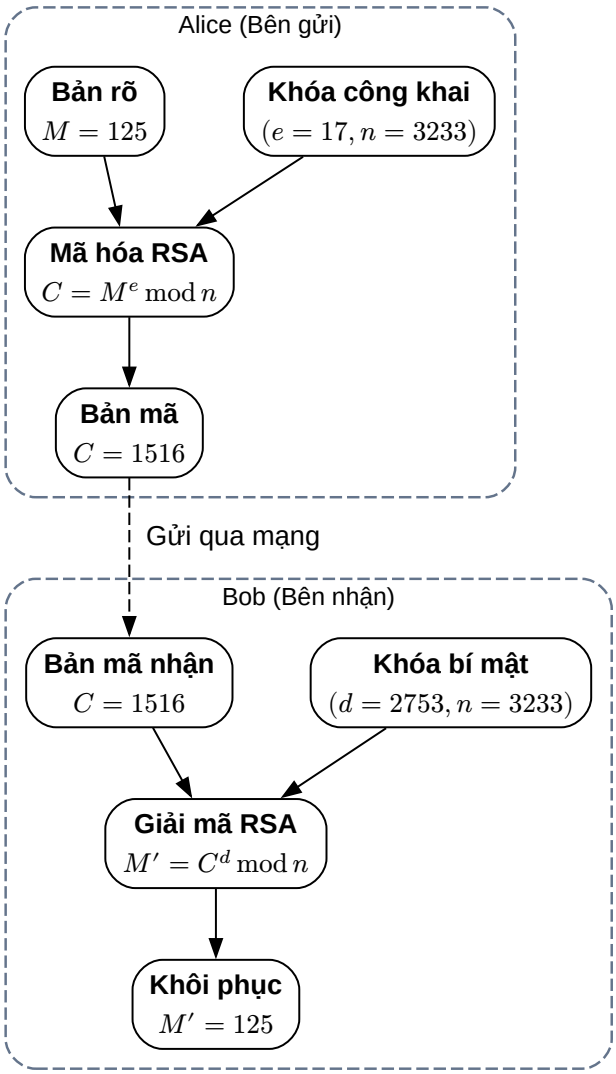
$$C = M^e \bmod n = 125^{17} \bmod 3233 = 1516$$

- **Giải mã (Bob thực hiện):**

$$M' = C^d \bmod n = 1516^{2753} \bmod 3233 = 125$$

Giai đoạn	Công thức	Kết quả tính toán
Bản rõ gốc	M	125
Bản mã	$C = M^e \bmod n$	1516
Bản giải mã	$M' = C^d \bmod n$	125 ($= M$)

7.3. Chuỗi chuyển đổi dữ liệu và khóa



Hình 3: Sơ đồ chu trình xử lý dữ liệu và khóa trong ví dụ số học RSA giữa Alice và Bob

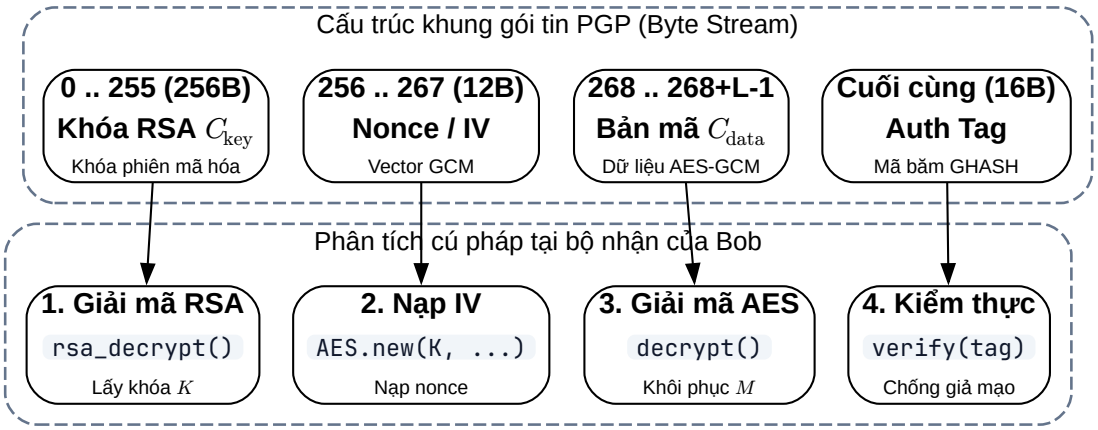
Quá trình giải mã cho kết quả $M' = 125$, trùng khớp hoàn toàn với giá trị gốc M, xác nhận tính toàn vẹn của cặp phép biến đổi $M^e \bmod n$ và $C^d \bmod n$.

8. Tích hợp RSA và AES trong Hybrid Encryption

Trong mô hình mã hóa lai, RSA không dùng để mã hóa trực tiếp khối dữ liệu bản rõ có kích thước lớn, mà chỉ mã hóa khóa phiên (Session Key). Toàn bộ khối dữ liệu thông điệp được mã hóa bằng thuật toán đối xứng AES để tối ưu hiệu năng tính toán.

8.1. Cấu trúc khung gói tin (Packet Framing & Offset)

Khi truyền tải qua mạng, dòng dữ liệu PGP được cấu trúc thành các trường liên tục với vị trí byte (Offset) và kích thước xác định, cho phép bộ giải mã ở phía nhận phân tích cú pháp (parse) chính xác:



Hình 4: Sơ đồ cấu trúc khung gói tin (Packet Framing) và luồng phân tích cú pháp tại bộ nhận của Bob

Trong cấu trúc gói tin trên, hai thành phần của chế độ mã hóa xác thực AES-GCM gồm có:

- **Vector khởi tạo (IV - Initialization Vector / Nonce):** Là chuỗi ngẫu nhiên 12 bytes (96 bits) được sinh mới cho mỗi thông điệp, nhằm tránh trùng lặp bản mã khi mã hóa các thông điệp khác nhau bằng cùng một khóa phiên. IV được truyền công khai trong gói tin, và theo chuẩn GCM, không được tái sử dụng cùng một cặp (K, IV).
- **Mã xác thực thông điệp (Authentication Tag):** Là thẻ kiểm thực mật mã 16 bytes (128 bits) do hàm băm GHASH tính toán từ dữ liệu mã hóa. Nếu dữ liệu trên đường truyền bị thay đổi, phép kiểm tra Authentication Tag tại phía nhận sẽ thất bại và gói tin bị từ chối.

8.2. Cơ sở toán học của AES-GCM (AEAD)

Chế độ AES-GCM kết hợp hai nền tảng số học để đạt được tính bảo mật và toàn vẹn song song:

1. **Mã hóa dòng CTR Mode:** Các khối bản rõ P_i được mã hóa qua phép XOR với chuỗi khóa giả ngẫu nhiên sinh bởi thuật toán AES:

$$C_i = P_i \oplus \text{AES}_K(\text{CB}_i)$$

với CB_i là giá trị Counter Block tính từ Nonce (12 bytes) nối với bộ đếm 32-bit tăng dần.

2. **Xác thực toàn vẹn bằng hàm băm GHASH:** Giá trị kiểm thực Authentication Tag được tính trên trường hữu hạn Galois $\mathbb{F}_{2^{128}}$ dựa trên đa thức bất khả quy:

$$f(x) = x^{128} + x^7 + x^2 + x + 1$$

Khóa băm xác thực $H = \mathrm{AES}_K(0^{128})$ được dùng để thực hiện phép nhân đa thức lặp Horner trên $\mathbb{F}_{2^{128}}$ qua từng khối bản mã C_i:

$$X_i = (X_{i-1} \oplus C_i) \cdot H$$

Giá trị Authentication Tag cuối cùng:

$$\mathrm{Tag} = X_m \oplus \mathrm{AES}_K(\mathrm{CB}_0)$$

Nếu kẻ tấn công can thiệp bất kỳ bit nào trên Ciphertext ($C_i \to C_i'$), giá trị băm X_m' sẽ biến đổi hoàn toàn theo tính chất khuếch tán của trường Galois, làm phép so sánh Tag tại phía Bob thất bại lập tức.

8.3. Phân công nhiệm vụ hai phía

Phía Alice (Mã hóa)	Phía Bob (Giải mã)
• Sinh Session Key ngẫu nhiên (128-bit). • Dùng AES-GCM mã hóa thông điệp bằng Session Key to tạo ra Ciphertext, Nonce và Authentication Tag. • Dùng Public Key RSA của Bob mã hóa Session Key to Encrypted Key. • Ghép 4 thành phần thành gói tin duy nhất gửi đi.	• Tiếp nhận toàn bộ gói tin từ Alice qua mạng. • Dùng Private Key RSA của Bob giải mã Session Key. • Dùng Session Key và Nonce giải mã Ciphertext bằng AES-GCM. • Đối chiếu Authentication Tag để kiểm tra tính toàn vẹn và chống giả mạo trước khi nhận thông điệp gốc.

8.4. Luồng điều khiển của mô-đun tích hợp

Tiến trình xử lý trong mã nguồn `algorithms/hybrid_pgp.py` được tổ chức thành hai luồng độc lập tương ứng với hai hàm nghiệp vụ:

1. **Hàm đóng gói mã hóa (`alice_encrypt_package`):**
   - Khởi tạo khóa phiên ngẫu nhiên K độ dài 128-bit (`os.urandom(16)`).
   - Mã hóa bản rõ M bằng AES-GCM với khóa K, trích xuất bộ ba: Ciphertext (C_{data}), Nonce (IV 12 bytes) và Authentication Tag (16 bytes).
   - Dùng Public Key của Bob (e_B, n) mã hóa khóa phiên K theo thuật toán RSA: $C_{\mathrm{key}} = K^{\{e_B\}} \bmod n$. Khóa phiên được mã hóa **trọn vẹn** dưới dạng số nguyên 128-bit (không lấy modulo n trước khi mã hóa), do đó yêu cầu modulus $n > 2^{128}$.
   - Ghép 4 thành phần thành cấu trúc payload duy nhất để truyền qua mạng.

2. **Hàm tiếp nhận và giải mã (`bob_decrypt_package`):**
   - Tách gói tin nhận được thành 4 thành phần theo cấu trúc offset đã định nghĩa ở Hình 4.
   - Dùng Private Key của Bob (d_B, n) giải mã trường C_{key} để khôi phục lại khóa phiên K. Kết quả số học được chuyển về đúng 16 bytes; nếu giải mã bằng sai khóa khiến giá

trị vượt quá 128-bit, phần dư được cắt bằng mask `& ((1 << 128) - 1)` thay vì ném `OverflowError`, nhờ đó AES-GCM vẫn kiểm tra Tag và từ chối gói tin một cách an toàn.

- Nạp K, Nonce và Authentication Tag vào bộ giải mã AES-GCM.
- Thực hiện giải mã và kiểm tra mã xác thực toàn vẹn: nếu dữ liệu bị thay đổi, ngoại lệ `ValueError` lập tức được kích hoạt để hủy thông điệp; ngược lại giải mã thành công bản rõ M.

8.5. Mã nguồn tích hợp Hybrid PGP (`algorithms/hybrid_pgp.py`)

Python

```python
"""
Mô-đun tích hợp Hybrid Encryption theo mô hình PGP (RSA + AES-GCM).
- Phía Alice: Sinh Session Key 128-bit ngẫu nhiên, mã hóa dữ liệu qua AES-GCM,
  mã hóa trọn vẹn Session Key qua RSA của Bob và đóng gói package.
- Phía Bob: Dùng Private Key RSA giải mã khôi phục Session Key, giải mã AES-GCM và đối chiếu Tag.
"""

import os
from .rsa import rsa_encrypt, rsa_decrypt
from .aes_gcm import aes_gcm_encrypt, aes_gcm_decrypt


def alice_encrypt_package(plaintext_str, bob_pub_key):
    """
    Quy trình đóng gói phía Alice.
    Trả về: (package_dict, session_key_raw)
    """
    session_key = os.urandom(16)  # Khóa phiên AES-128 (16 bytes = 128 bits)
    plaintext_bytes = plaintext_str.encode("utf-8")

    ciphertext, nonce, auth_tag = aes_gcm_encrypt(session_key, plaintext_bytes)

    # Mã hóa trọn vẹn Session Key 16 bytes bằng RSA Public Key của Bob
    key_int = int.from_bytes(session_key, byteorder="big")
    enc_session_key = rsa_encrypt(bob_pub_key, key_int)

    package = {
        "enc_session_key": enc_session_key,
        "nonce": nonce,
        "ciphertext": ciphertext,
        "auth_tag": auth_tag,
    }
    return package, session_key


def bob_decrypt_package(package, bob_priv_key):
    """
    Quy trình tiếp nhận và giải mã phía Bob.
```

```
    Ném ngoại lệ nếu xác thực Tag thất bại hoặc khóa không khớp.
    """
    enc_session_key = package["enc_session_key"]
    recovered_key_int = rsa_decrypt(bob_priv_key, enc_session_key)

    # Khôi phục trực tiếp 16 bytes Session Key từ kết quả giải mã RSA
    try:
        recovered_session_key = recovered_key_int.to_bytes(16, byteorder="big")
    except OverflowError:
        # Nếu C_key bị thay đổi hoặc giải mã bằng sai Private Key làm số nguyên > 16 bytes
        recovered_session_key = (recovered_key_int & ((1 << 128) - 1)).to_bytes(16, byteorder="big")

    plaintext_bytes = aes_gcm_decrypt(
        session_key=recovered_session_key,
        nonce_bytes=package["nonce"],
        ciphertext_bytes=package["ciphertext"],
        auth_tag_bytes=package["auth_tag"],
    )
    return plaintext_bytes.decode("utf-8")


if __name__ == "__main__":
    from .rsa import generate_rsa_keys

    bob_pub, bob_priv = generate_rsa_keys(bits=256)
    msg = "PGP Hybrid Encryption tích hợp thành công!"
    pkg, s_key = alice_encrypt_package(msg, bob_pub)
    decrypted = bob_decrypt_package(pkg, bob_priv)
    print("Kiểm thử tích hợp Hybrid PGP:")
    print(f"Session Key (16B): {s_key.hex()}")
    print(f"Bản rõ khôi phục : {decrypted}")
    assert msg == decrypted
    print("Khôi phục thành công.")
```

Lưu ý kỹ thuật và tính minh bạch:

- **Chế độ AES-GCM:** Sử dụng **Nonce** (Number used once) nhằm triệt tiêu hiện tượng trùng lặp khối mã và sử dụng **Authentication Tag** (128-bit) để bảo đảm toàn vẹn dữ liệu.
- **Ranh giới triển khai:** Nhóm tự lập trình thuật toán số học RSA để nắm vững nguyên lý toán học, đồng thời tích hợp thư viện AES chuẩn công nghiệp (`Cryptodome`) để bảo đảm hiệu năng và tính an toàn mật mã.

9. Demo chương trình và kết quả kiểm thử

Để kiểm tra tính đúng đắn và khả năng phát hiện lỗi của chương trình Python (`src/hybrid_demo.py`), hệ thống được thử nghiệm qua 3 kịch bản:

Kiểm thử	Kịch bản thử nghiệm	Kết quả quan sát & Phân tích
Test 01	Bob sử dụng đúng Private Key tương ứng với Public Key của mình.	Khôi phục thành công 100% nội dung thông điệp gốc ban đầu. Khóa phiên và dữ liệu được giải mã chính xác.
Test 02	Kẻ tấn công xen vào giữa (Man-in-the-Middle) can thiệp, đảo 1 bit trong Ciphertext trước khi Bob giải mã.	Phát hiện can thiệp: Thư viện bắt lỗi ngoại lệ `ValueError: MAC check failed` . Bản tin bị hủy bỏ ngay lập tức.
Test 03	Giải mã bằng một Private Key không khớp (khóa của bên thứ ba Eve).	Thất bại: Phép tính RSA cho ra giá trị Session Key sai lệch, dẫn tới giải mã AES thất bại và ném ngoại lệ.

9.1. Kết quả thực thi trên terminal

Dưới đây là nhật ký xuất từ terminal khi chạy `python3 src/hybrid_demo.py` :

```
1  ================================================================
2  DEMO PGP HYBRID ENCRYPTION (RSA + AES-GCM)
3  ================================================================
4
5  [ALICE] Thông điệp gốc: "Báo cáo tiến độ Nhóm 7: PGP Hybrid Encryption hoạt động
   chính xác!"
6  [ALICE] Mã hóa hoàn tất:
7    + Encrypted Session Key (RSA) : 0xbe013053248c3eb35cee26cd3ec7e6ae2c...
8    + Nonce (AES-GCM)             : 0398e089ee4da8b197573906d2cc0cd5
9    + Ciphertext (AES)            :
   a558ef5f7a0d67633054f9e7e588c7d8988db1c9f2a8a90e... (81 bytes)
10   + Authentication Tag          : a8d0cf9ba50332bc0b175330d498275a
11
12 ----------------------------------------------------------------
13 TEST CASE 01: Bob dùng đúng Private Key của mình
14 [BOB] Giải mã thành công: "Báo cáo tiến độ Nhóm 7: PGP Hybrid Encryption hoạt động
   chính xác!"
15 >>> KẾT QUẢ: [PASSED] Khôi phục chính xác 100% dữ liệu.
16
17 ----------------------------------------------------------------
18 TEST CASE 02: Kẻ tấn công sửa đổi 1 byte trong Ciphertext
19 [BOB] CẢNH BÁO AN TOÀN: Phát hiện dữ liệu bị can thiệp!
20       Chi tiết ngoại lệ: MAC check failed (Authentication Tag không khớp)
21 >>> KẾT QUẢ: [PASSED] Chặn thành công tấn công sửa đổi bản mã.
```

```
22
23 ----------------------------------------------------------------
24 TEST CASE 03: Giải mã bằng Private Key không hợp lệ (Khóa của Eve)
25 [EVE] Thất bại: Giải mã RSA ra Session Key sai -> AES-GCM từ chối xác thực.
26       Chi tiết ngoại lệ: MAC check failed (Sai khóa phiên dẫn đến kiểm thực thất
         bại)
27 >>> KẾT QUẢ: [PASSED] Từ chối truy cập trái phép khi sai khóa.
28 ================================================================
```

Nhận xét kết quả kiểm thử:

Ba tình huống kiểm thử trên cho thấy mô hình kết hợp RSA và AES-GCM đáp ứng tính bí mật của nội dung thông điệp trên đường truyền, đồng thời phát hiện được các can thiệp làm sai lệch dữ liệu.

10. Mô phỏng truyền thông PGP Hybrid qua Socket TCP

Các chương trước đã kiểm chứng mô hình mã hóa lai trong phạm vi một tiến trình duy nhất (`src/hybrid_demo.py`). Để tiến gần hơn tới bài toán thực tế của môn học Mạng máy tính, nhóm bổ sung một bản demo truyền thông qua giao thức **TCP Socket**, trong đó Alice và Bob có thể chạy trên hai máy tính khác nhau trong cùng mạng LAN.

10.1. Kiến trúc chương trình

Mã nguồn được tách thành 3 tệp độc lập trong thư mục `PGP_RSA_AES_Socket_3_files/`:

Tệp nguồn	Vai trò
`server_bob.py`	Máy B — TCP Server: sinh cặp khóa RSA bằng `algorithms/rsa.py`, lắng nghe tại `0.0.0.0:5000`, nhận gói tin và giải mã.
`client_alice.py`	Máy A — TCP Client: lấy Public Key của Bob, gọi `encrypt_message`, gửi gói tin qua TCP và hiển thị phản hồi.
`pgp_crypto.py`	Cầu nối dữ liệu: chuyển đổi giữa cấu trúc JSON/Base64 trên đường truyền và các hàm thuần Python trong `algorithms/hybrid_pgp.py`.

Cả hai phía đều import trực tiếp gói `algorithms` của nhóm, nhờ đó toàn bộ thuật toán Euclid mở rộng, RSA và AES-GCM chỉ tồn tại ở một nơi duy nhất, tránh sai lệch giữa bản demo cục bộ và bản chạy qua mạng.

10.2. Đóng gói và truyền nhận gói tin

TCP là giao thức luồng byte (byte stream), không bảo toàn ranh giới thông điệp. Vì vậy chương trình áp dụng cơ chế **length-prefixed framing**: mỗi gói tin được gửi kèm 4 byte độ dài ở đầu, theo thứ tự Big-Endian.

```python
def send_json(connection, payload_dict):
    data = json.dumps(payload_dict, separators=(",", ":")).encode("utf-8")
    connection.sendall(struct.pack("!I", len(data)) + data)

def receive_json(connection):
    packet_length = struct.unpack("!I", recv_exact(connection, 4))[0]
    if not 0 < packet_length <= MAX_PACKET:
        raise ValueError("Kích thước gói tin không hợp lệ.")
    packet = recv_exact(connection, packet_length)
    return json.loads(packet.decode("utf-8"))
```

Hàm `recv_exact` lặp `recv()` cho tới khi đọc đủ số byte yêu cầu, khắc phục hiện tượng một lần `recv()` chỉ trả về một phần dữ liệu. Các trường nhị phân (`nonce`, `ciphertext`, `auth_tag`)

được mã hóa Base64, còn `enc_session_key` được biểu diễn dạng chuỗi hex vì là số nguyên lớn. Gói tin bị giới hạn tối đa 1 MB và mỗi kết nối có timeout 15 giây.

10.3. Bắt tay và xác thực Public Key

Trước khi gửi dữ liệu, Alice cần Public Key (e_B, n) của Bob. Chương trình hỗ trợ hai đường lấy khóa:

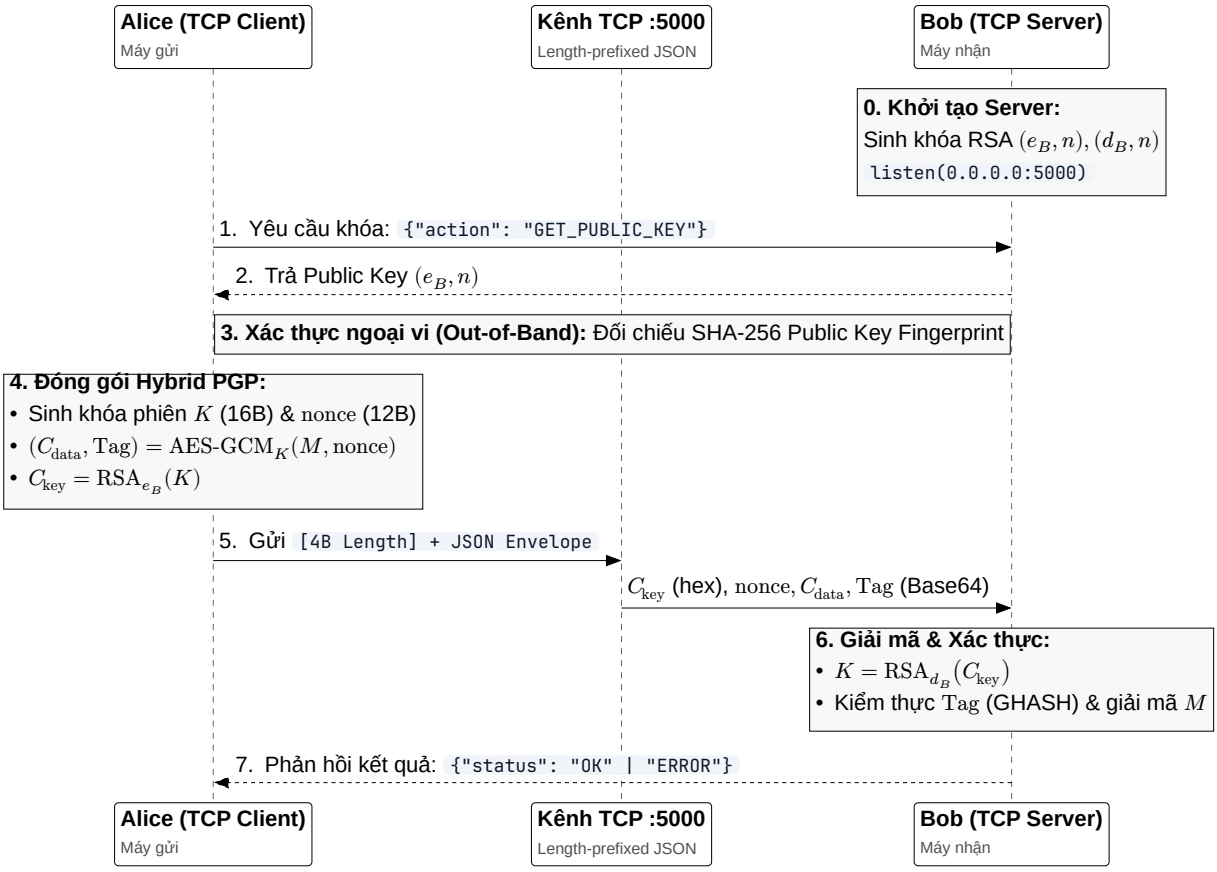
1. **Đọc từ tệp cục bộ** `bob_public.json` nếu đã được sao chép từ máy Bob.
2. **Tải tự động qua mạng**: Alice gửi yêu cầu `{"action": "GET_PUBLIC_KEY"}`, Bob phản hồi `{"e": ..., "n": ...}`, Alice lưu lại thành tệp cho các lần sau.

Sau khi có khóa, cả hai máy in ra **mã băm SHA-256** của Public Key:

```python
def fingerprint(public_key):
    e, n = public_key
    return hashlib.sha256(f"RSA:{e}:{n}".encode("ascii")).hexdigest()
```

Người dùng đối chiếu trực tiếp hai chuỗi hex trên màn hình hai máy. Nếu trùng khớp, khóa nhận được đúng là của Bob; nếu khác, kênh đã bị can thiệp và phải dừng phiên. Đây là bước kiểm tra ngoài kênh (out-of-band) nhằm hạn chế tấn công xen giữa ở giai đoạn trao đổi khóa.

10.4. Quy trình hoạt động



Hình 5: Sơ đồ tuần tự mô phỏng PGP Hybrid qua Socket TCP giữa máy Alice và máy Bob

10.5. Kịch bản kiểm thử trên mạng

Giao diện dòng lệnh của Alice cung cấp 3 chế độ tương ứng với 3 tình huống kiểm thử:

Chế độ	Thao tác	Kỳ vọng
`1`	Gửi gói tin hợp lệ, không chỉnh sửa.	Bob giải mã đúng bản rõ và trả về `status: OK`.
`2`	Đảo bit byte đầu tiên của `ciphertext` trước khi gửi.	AES-GCM báo `MAC check failed`; Bob hủy gói tin và trả về `status: ERROR`.
`3`	XOR `enc_session_key` với `0x1` để làm sai khóa phiên.	RSA cho ra Session Key sai; AES-GCM từ chối xác thực, Bob trả về `status: ERROR`.

10.6. Hướng dẫn chạy

Khởi động máy Bob trước; màn hình sẽ in địa chỉ IP LAN cần nhập ở máy Alice:

```bash
cd PGP_RSA_AES_Socket_3_files
python3 server_bob.py
```

Trên máy Alice, truyền IP của Bob làm tham số (dùng `127.0.0.1` nếu chạy cùng máy):

```bash
cd PGP_RSA_AES_Socket_3_files
python3 client_alice.py 192.168.1.11
```

Nếu chạy trên cùng một máy, cần mở hai cửa sổ terminal riêng biệt.

10.7. Kết quả thực nghiệm

Nhật ký phía Alice khi gửi gói tin hợp lệ (chế độ `1`):

```
[ALICE] SHA-256 Public Key Fingerprint của Bob:
        2acda677ab4ca7b3948733f742a6630ae7feec0fe8659ad5e3e01add128ac796
[ALICE] [1] Đã sinh Session Key K (16B)  : 4ca6cc7cc3807b853e8951d8b61ec716
[ALICE] [2] enc_session_key (RSA)        :
0x1881420e6080945c46b52904d59d421efb601dceb7...
[ALICE] [3] nonce (AES-GCM 12B)          : YJMcoCbdoqQAZgFL
[ALICE] [4] auth_tag (GHASH 16B)         : rAO87+vJhSC+3TlTKLbAcQ==
[ALICE] [5] ciphertext (AES)             : 7IA1Msw+8ysC64slujE3txcHOUnns/
i4V0L9YuUnQRdv...
[ALICE] >>> Đã truyền 309 bytes qua TCP tới 127.0.0.1:5000.
[ALICE] <<< Phản hồi từ máy Bob: [OK] Bob da giai ma va xac thuc thanh cong!
```

Nhật ký tương ứng phía Bob:

```
 [BOB] Đã sinh mới cặp khóa RSA 511-bit từ algorithms.rsa.
 [BOB] Đang lắng nghe TCP tại 0.0.0.0:5000 ... (Nhấn Ctrl+C để dừng)

```

```
[BOB] >>> Kết nối mới từ 127.0.0.1:43994
[BOB] Đã nhận gói tin PGP gồm 4 thành phần:
      + enc_session_key (RSA) : 0x1881420e6080945c46b52904d59d421efb601dceb7...
      + nonce (AES-GCM 12B)   : YJMcoCbdoqQAZgFL
      + auth_tag (GHASH 16B)  : rAO87+vJhSC+3TlTKLbAcQ==
      + ciphertext (AES)      : 7IA1Msw+8ysC64slujE3txcHOUnns/i4V0L9YuUnQRdv...
[BOB] Đã giải mã RSA & xác thực toàn vẹn AES-GCM thành công!
[BOB] >>> BẢN RÕ NHẬN ĐƯỢC: "Thong diep kiem thu PGP Hybrid qua TCP!"
```

Khi chọn chế độ `2` hoặc `3`, phía Bob phát hiện bất thường và hủy gói tin:

```
[BOB] [!] CẢNH BÁO AN TOÀN MẬT MÃ:
      Phát hiện gói tin bị can thiệp hoặc sai khóa! Chi tiết: MAC check failed
      -> Bản tin đã bị hủy bỏ theo cơ chế toàn vẹn AES-GCM.
```

Plain Text

Phía Alice nhận phản hồi `[ERROR] Tu choi goi tin: MAC check failed`, xác nhận gói tin đã bị chặn đúng như thiết kế.

Giới hạn của bản mô phỏng:

- Kênh TCP trong demo **không được bọc TLS**. Do đó bước tải Public Key tự động vẫn có thể bị tấn công xen giữa; việc đối chiếu fingerprint thủ công chỉ là biện pháp giảm thiểu ở mức minh họa.
- Khóa RSA 256-bit (mặc định `bits=256`) chỉ đủ chứa Session Key 128-bit và phục vụ mục tiêu học tập. RFC 8017 yêu cầu khóa tối thiểu 2048-bit cùng đệm RSA-OAEP cho triển khai thực tế.
- Giao thức chưa có cơ chế chống phát lại (replay), chưa xác thực danh tính bằng chứng chỉ số và chưa hỗ trợ nhiều phiên đồng thời.

11. Kết quả đạt được và hướng phát triển

Dưới đây là tổng hợp tiến độ nghiên cứu của nhóm cùng lộ trình hoàn thiện cho đồ án trong giai đoạn tiếp theo:

11.1. Tổng kết khối lượng công việc

Đã thực hiện	Hướng phát triển tiếp theo
• Nắm vững cơ sở lý thuyết về mô hình Hybrid Encryption và cấu trúc chuẩn PGP/ OpenPGP. • Tự lập trình giải thuật RSA toán học bằng Python (Euclid mở rộng, tính modulo). • Cài đặt sinh khóa RSA động với kiểm tra nguyên tố Miller–Rabin, thay thế cặp p, q cố định. • Minh họa thành công chu trình mã hóa và giải mã số học RSA. • Xây dựng chương trình Hybrid tích hợp RSA – AES và hoàn thành bộ 3 kiểm thử cơ bản. • Mô phỏng truyền thông PGP Hybrid qua Socket TCP giữa hai máy, có đóng khung gói tin và đối chiếu fingerprint SHA-256.	• Bọc kênh truyền bằng TLS và thay cơ chế trao đổi khóa thủ công bằng chứng chỉ số. • Mở rộng chức năng tạo chữ ký số bằng RSA và hàm băm SHA-256 để đảm bảo tính chống chối bỏ. • Kiểm thử hệ thống với dữ liệu thực tế lớn (tệp tin văn bản, ảnh, nhị phân). • Đo đạc thời gian thực thi (độ trễ và thông lượng) giữa mã hóa thuần RSA so với Hybrid Encryption.

11.2. Kế hoạch triển khai giai đoạn tới

Thời gian	Nhiệm vụ trọng tâm	Kết quả dự kiến đầu ra
Tuần 1	Hoàn thiện module Chữ ký số	Tích hợp RSA + SHA-256 để ký và xác thực chữ ký điện tử.
Tuần 2	Thực nghiệm tệp tin dung lượng lớn	Xử lý mã hóa/giải mã các file nhị phân, hình ảnh, tài liệu PDF.
Tuần 3	Đo đạc Benchmark & Hiệu năng	Biểu đồ so sánh thời gian thực thi giữa RSA thuần và Hybrid Encryption.
Tuần 4	Hoàn thiện báo cáo & Slide bảo vệ	Đóng gói mã nguồn hoàn chỉnh và tài liệu báo cáo nghiệm thu.

Định vị phạm vi đề tài:

Phạm vi hiện tại: Chương trình là bản mô phỏng trực quan cơ chế mã hóa lai RSA – AES phục vụ mục tiêu học tập và nghiên cứu cấu trúc PGP. Nhóm chưa đồng nhất bản demo này với một phần mềm OpenPGP hoàn chỉnh (như GnuPG), vốn đòi hỏi định dạng gói dữ liệu RFC 4880, hệ thống chứng thực Web of Trust và cơ chế quản lý khóa an toàn trên đĩa cứng.

TÀI LIỆU THAM KHẢO

[1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th a.b. Pearson, 2017.

[2] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC”, NIST Special Publication 800–38D, 2007.

[3] K. Moriarty, B. Kaliski, J. Jonsson, và A. Rusch, “PKCS #1: RSA Cryptography Specifications Version 2.2”, RFC 8017, 2016.

[4] J. Callas, L. Donnerhacke, H. Finney, D. Shaw, và R. Thayer, “OpenPGP Message Format”, RFC 4880, 2007.

[5] P. Wouters và D. Huigens, “OpenPGP”, RFC 9580, 2024.

[6] R. L. Rivest, A. Shamir, và L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems”, *Communications of the ACM*, tập 21, số 2, tr 120–126, 1978.